

Your agents write the fix.

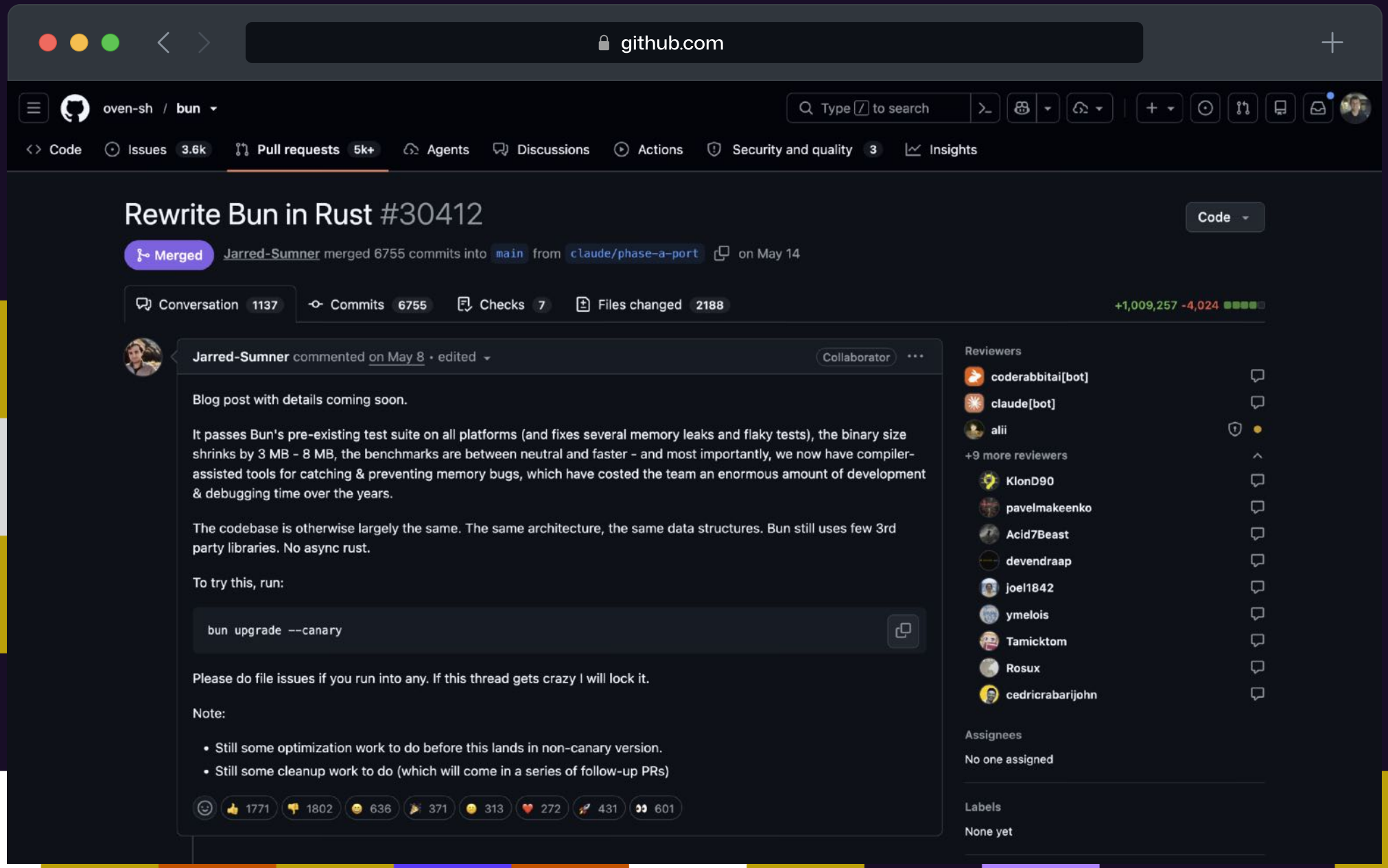
Observability supplies the context.



Michael Beemer
Product Manager
Bluebox by Dynatrace

AI made writing code faster.

Releasing has become the bottleneck.


github.com
oven-sh / bun
Type / to search
Code
Issues 3.6k
Pull requests 5k+
Agents
Discussions
Actions
Security and quality 3
Insights
Rewrite Bun in Rust #30412
Code
Merged
Jarred-Sumner merged 6755 commits into main from claude/phase-a-port on May 14
Conversation 1137
Commits 6755
Checks 7
Files changed 2188
+1,009,257 -4,024
Jarred-Sumner commented on May 8 • edited
Collaborator
Blog post with details coming soon.
It passes Bun's pre-existing test suite on all platforms (and fixes several memory leaks and flaky tests), the binary size shrinks by 3 MB - 8 MB, the benchmarks are between neutral and faster - and most importantly, we now have compiler-assisted tools for catching & preventing memory bugs, which have costed the team an enormous amount of development & debugging time over the years.
The codebase is otherwise largely the same. The same architecture, the same data structures. Bun still uses few 3rd party libraries. No async rust.
To try this, run:
bun upgrade --canary
Please do file issues if you run into any. If this thread gets crazy I will lock it.
Note:
Still some optimization work to do before this lands in non-canary version.
Still some cleanup work to do (which will come in a series of follow-up PRs)
1771
1802
636
371
313
272
431
601
Reviewers
coderabbitai[bot]
claude[bot]
alii
+9 more reviewers
KlonD90
pavelmakeenko
Acid7Beast
devendraap
joel1842
ymelois
Tamicktom
Rosux
cedricrabarijohn
Assignees
No one assigned
Labels
None yet

WHY THIS MATTERS

Faster, bigger, more often

+26%

More completed tasks for developers using an AI assistant

MANAGEMENT SCIENCE · 3 FIELD EXPERIMENTS · 4,867 DEVS

+23%

More PRs merged per month on GitHub, 2025 vs 2024

GITHUB OCTOVERSE 2025

1 in 3

Pull requests on GitHub now involve an agent

MICROSOFT FY26 Q4 EARNINGS · JUL 2026

WHY THIS MATTERS

Reviewing got harder

+51%

Larger PRs on teams with high AI adoption

FAROS AI · Q2 2026 · 22K DEVS

5×

Longer median review time on those teams

FAROS AI · Q2 2026 · 22K DEVS

WHY THIS MATTERS

The agent sees the repo, not the world



What it sees

The code, the tests, the docs. In other words, intent.



What it doesn’t

Real traffic, real data, real customers. In other words, behavior.

How do we lower the risk?

And when something breaks, how do we find it fast?

Session Agenda

01 A crash course in OpenTelemetry

02 From signals to answers

03 Demo: from “it’s slow” to a verified fix

04 You still own what you ship

A crash course in OpenTelemetry

Four signals



Metrics

That something is wrong, in aggregate.



Traces

Why a request was slow, step by step.



Logs / Events

What happened, linked by trace ID.



Profiles

Where CPU and memory go, in code.

Metrics tell you that something is wrong



Aggregates

Counts, rates, and percentiles over time.



Cheap and familiar

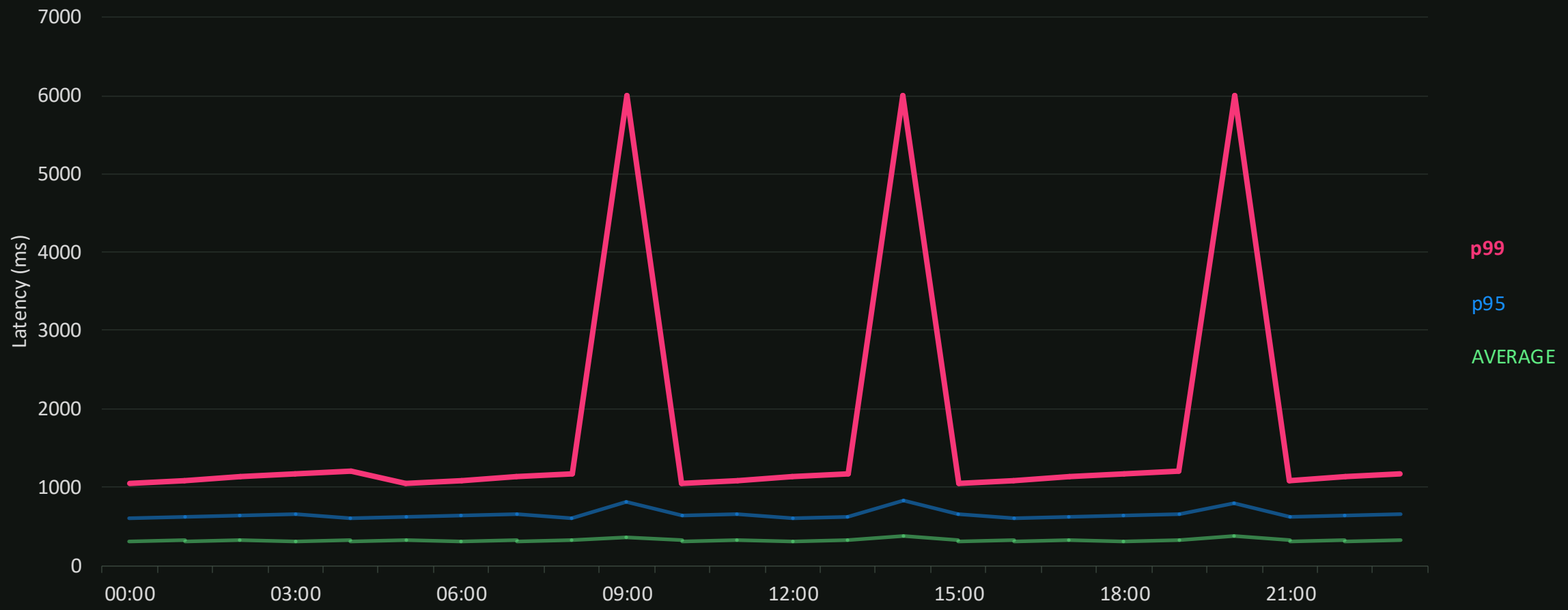
Small, fast, and made for dashboards and alerts.



But blurry

An anomaly says something is off, not what, who, or why.

Averages don’t tell the full story



Logs are detailed, but isolated

Granular, but alone: which request? What happened before it?

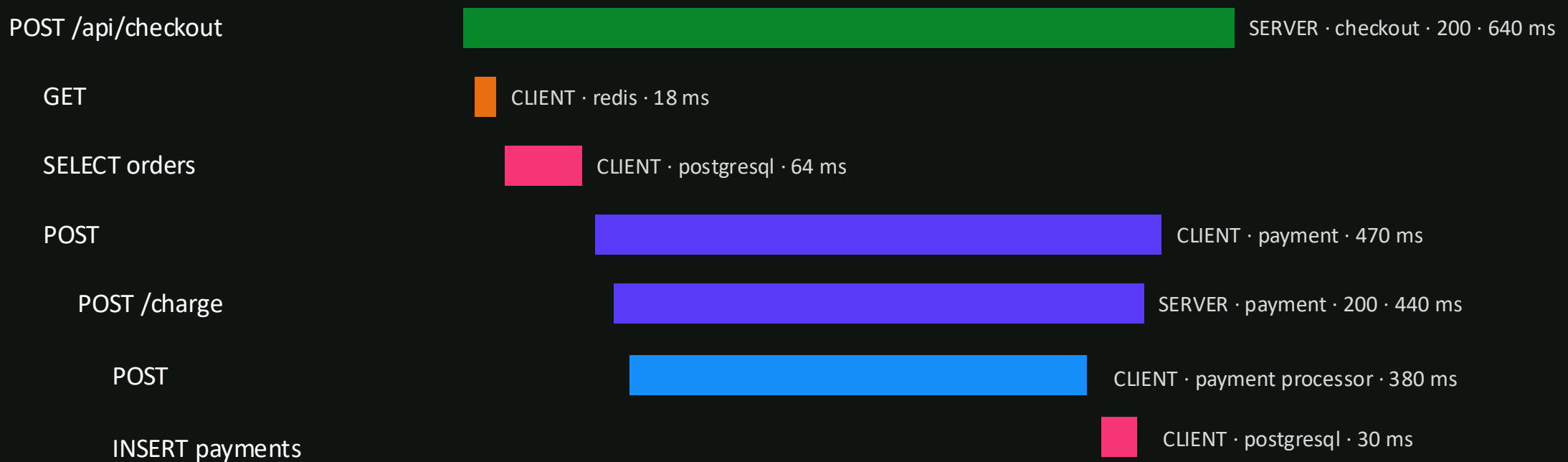
```
14:02:11.482  WARN  checkout slow: 3412 ms  account.id=acct_5pzc28h7
```

With a trace ID, one click takes you to the whole request.

```
14:02:11.482  WARN  checkout slow: 3412 ms  account.id=acct_5pzc28h7
trace_id=4bf92f3577b34da6a3ce929d0e0e4736
```

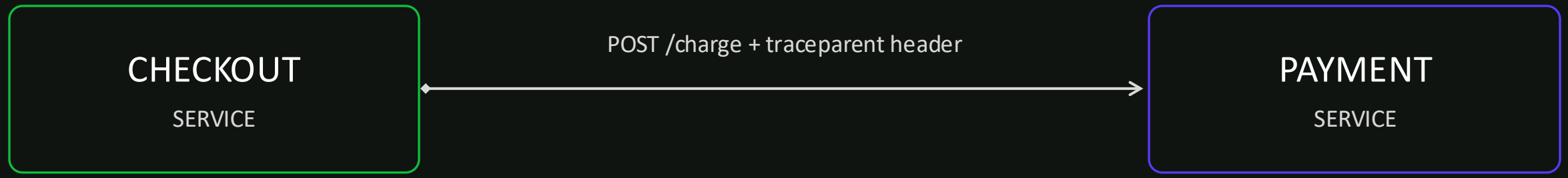
METRICS SAY SOMETHING IS WRONG · LOGS SAY WHAT HAPPENED · TRACES CONNECT THEM

Anatomy of a trace



A trace is a collection of spans grouped with a trace ID.

Tracing across services

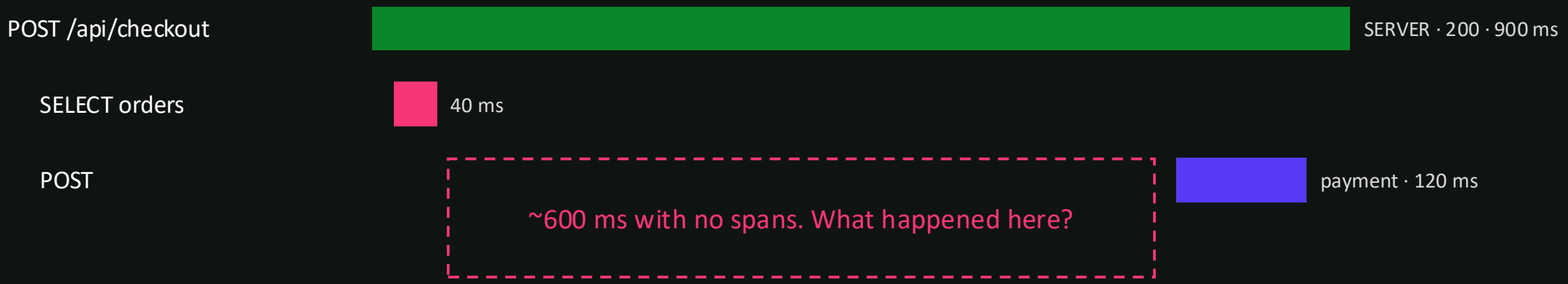


traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01

VERSION | TRACE ID (SHARED BY EVERY SPAN) | PARENT SPAN ID | FLAGS

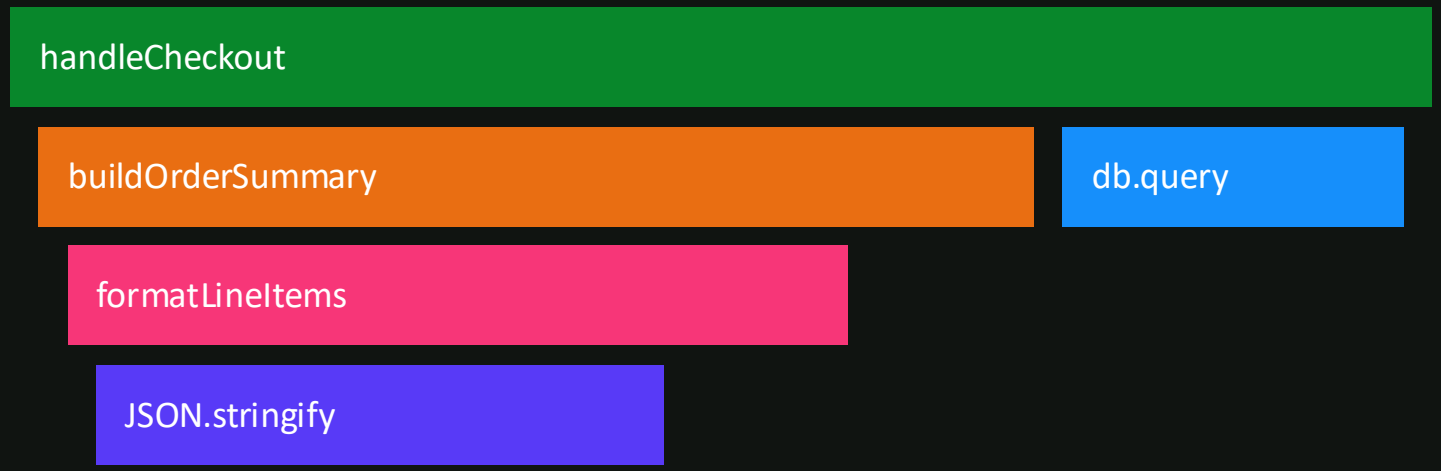
Caches, databases, and third-party APIs usually don’t continue the trace. They appear as client spans from the service that called them.

Spans can have gaps



Instrumentation usually wraps I/O: HTTP calls, queries, queues. JSON parsing, loops, and serialization in between leave no trace.

Profiles fill some gaps, not all



FLAME GRAPH · WIDER = MORE CPU TIME

Strengths

Thread-level snapshots in prod

Limitations

Sampled, aggregated, and deduped

Each signal sees part of the picture. Something has to connect them back to the code.

Great signals. Missing context.

METRICS	**Is something wrong?**
LOGS	**What happened?**
TRACES	**Where, across services?**
PROFILES	**Which function?**
MISSING	**Who was affected, and which code changed?**

From signals to answers

Tie observability to code

middleware.ts

```
OTEL_SERVICE_NAME=checkout
OTEL_RESOURCE_ATTRIBUTES=
  deployment.environment.name=production,
  vcs.repository.url.full=github.com/beeme1mr/shop-checkout,
  vcs.ref.head.revision=${GIT_SHA}
```

Add business context

middleware.ts

```ts
import { Request, Response, NextFunction } from "express";
import { trace } from "@opentelemetry/api";

/**
 * Middleware to enrich the current OpenTelemetry span with account details.
 */
export function accountTracingMiddleware(
  req: Request,
  res: Response,
  next: NextFunction,
): void {
  // 1. Retrieve the active span from the current context
  const activeSpan = trace.getActiveSpan();

  if (activeSpan) {
    // 2. Extract account details (typically populated by an auth middleware)
    const accountId = req.user?.accountId || "anonymous";
    const accountTier = req.user?.tier || "free";

    // 3. Set standard OpenTelemetry attributes
    activeSpan.setAttribute("account.id", accountId);
    activeSpan.setAttribute("account.tier", accountTier);
  }

  next();
}
```

Enter the observability agent



Your coding agent understands the code. An observability agent understands what it does in production.

Bluebox is your observability agent



Instruments

Guides you through adding OpenTelemetry to your services.



Understands

Learns how your system runs in production: topology, dependencies, and health.



Investigates

Connects telemetry to the code that caused it, and hands the evidence to your coding agent.



Automates

Watches for anomalies and antipatterns, like N+1 queries, and starts an investigation on its own.

From “it’s slow” to a verified fix

A customer reaches out

SUPPORT · TICKET #4821 · MEGASTREAM LTD

S

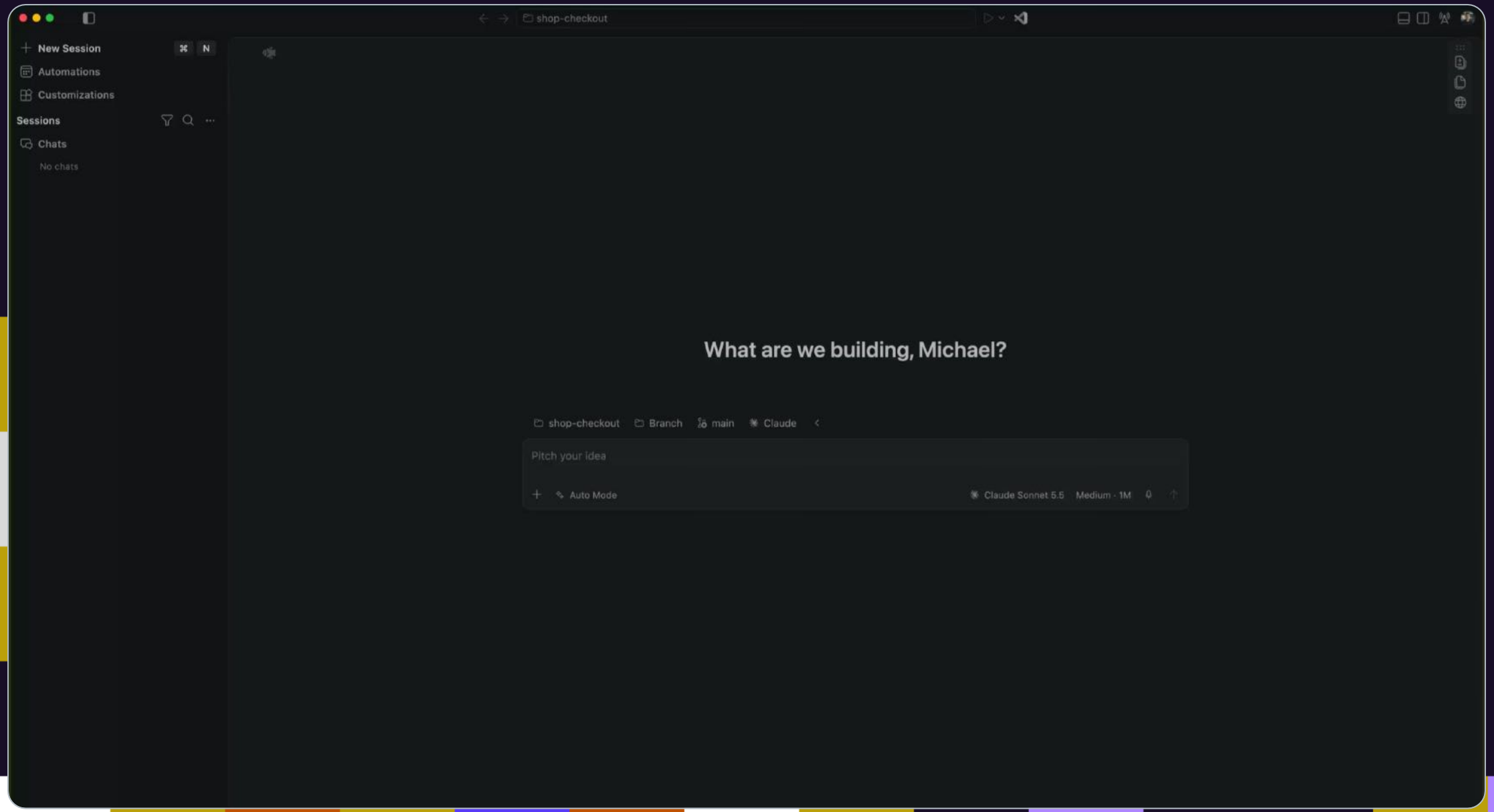
Sam · MegaStream Ltd 2:14 PM

Hi! Checkout has been really slow for us today. Is something going on?

Support 2:16 PM

Thanks, Sam. Looking into it now.

?


shop-checkout
New Session
N
Automations
Customizations
Sessions
Chats
No chats
What are we building, Michael?
shop-checkout
Branch
main
Claude
Pitch your idea
Auto Mode
Claude Sonnet 5.5
Medium · 1M

From a vague question to a fix

Copilot asks Bluebox why MegaStream is slow

Run MCP tool bluebox__list_services

Run MCP tool bluebox__ask

Input

```
{
  "question": "MegaStream reports slow checkout.
  Is checkout latency elevated for the MegaStream
  account compared to other accounts? What are
  the slowest spans and the likely cause?",
  "entity_id": "SERVICE-EEAC6E4EA9F5C55B",
  "since": "24h"
}
```

Under the hood: the calls to Bluebox

1 · LATENCY PERCENTILES POST /api/checkout, last hour	→ Global p99 332 ms. Looks healthy.
2 · SPLIT BY ACCOUNT.ID p99 per account	→ MegaStream: 3.45 s. Everyone else at or under 419 ms.
3 · SLOWEST TRACE for that account	→ One orders query, then ~1,200 sequential selects.
4 · CODE LOCATION at the running commit	→ src/orders.ts @ 115bbd1: a query inside a loop.

How the agent reasoned

“Checkout is slow…”	p95 and p99 of POST /api/checkout, not the average
“…for MegaStream.”	Slice by account.id: one account is far slower than the rest
“Why?”	That account’s traces: one query per order, ~1,200 in a row
“Where in the code?”	vcs.ref.head.revision and the repo: the loop in src/orders.ts

Bluebox responded with evidence-based answers

One account, `acct_5pzc28h7` (enterprise tier), has far higher checkout latency than all others. I couldn't confirm it's MegaStream, because no span or log carries that name. Its typical checkout takes about **3.1 s**; other accounts are well under half a second.

Evidence

- **Comparison:** For `POST /api/checkout` over 24 hours, this account has p50 ≈ **3,060 ms**, p95 ≈ **3,775 ms**, and a worst request of **22 s**. The next-slowest enterprise accounts have p50 of 270–340 ms and p95 of 350–430 ms. All other accounts combined have p50 78 ms and p95 224 ms.
- **Not failures:** All requests returned HTTP 201, just slowly.
- **Slowest spans:** The three worst (22.1 s, 16.3 s, 16.2 s) are checkout request spans from 17:31 UTC today. Trace IDs: `c31e3b51c394d50890e5e3208cb463bf`, `9c32be8bc00489318bb9f1af567d963b`, `31acc17c3bf6c5aedae0dda1f74bb42c`.
- **Not a regression:** 571 requests since 17:07 UTC, with no exceptions, no version difference (all `2.0.0`), and no pattern change. The slowness looks constant.

Recommended action

- **Confirm the account:** Check whether `acct_5pzc28h7` is MegaStream's ID. If not, give me the right one and I'll re-run the comparison.
- **Find the cause:** Open trace `c31e3b51c394d50890e5e3208cb463bf` and review account-specific checkout logic (enterprise integration, rate-limit or retry loop, pre-DB call). Adding spans there would pinpoint the ~3 s step.

Copilot confirm in code

MegaStream's checkout is slow because of an N+1 query in `getOutstandingTotal`, in `src/orders.ts:13-23`. I haven't changed any code.

Confirmed from the DB: MegaStream Ltd is `acct_5pzc28h7`, an enterprise account with 1,200 open orders.

Impact (per-account query, `npm run query:accounts`):

- MegaStream's p50 is about 3.5 s and p99 about 4.3 s.
- Every other account has a p99 of 540 ms or less, and the global p99 is 409 ms.
- Requests succeed with HTTP 201, just slowly.

Cause:

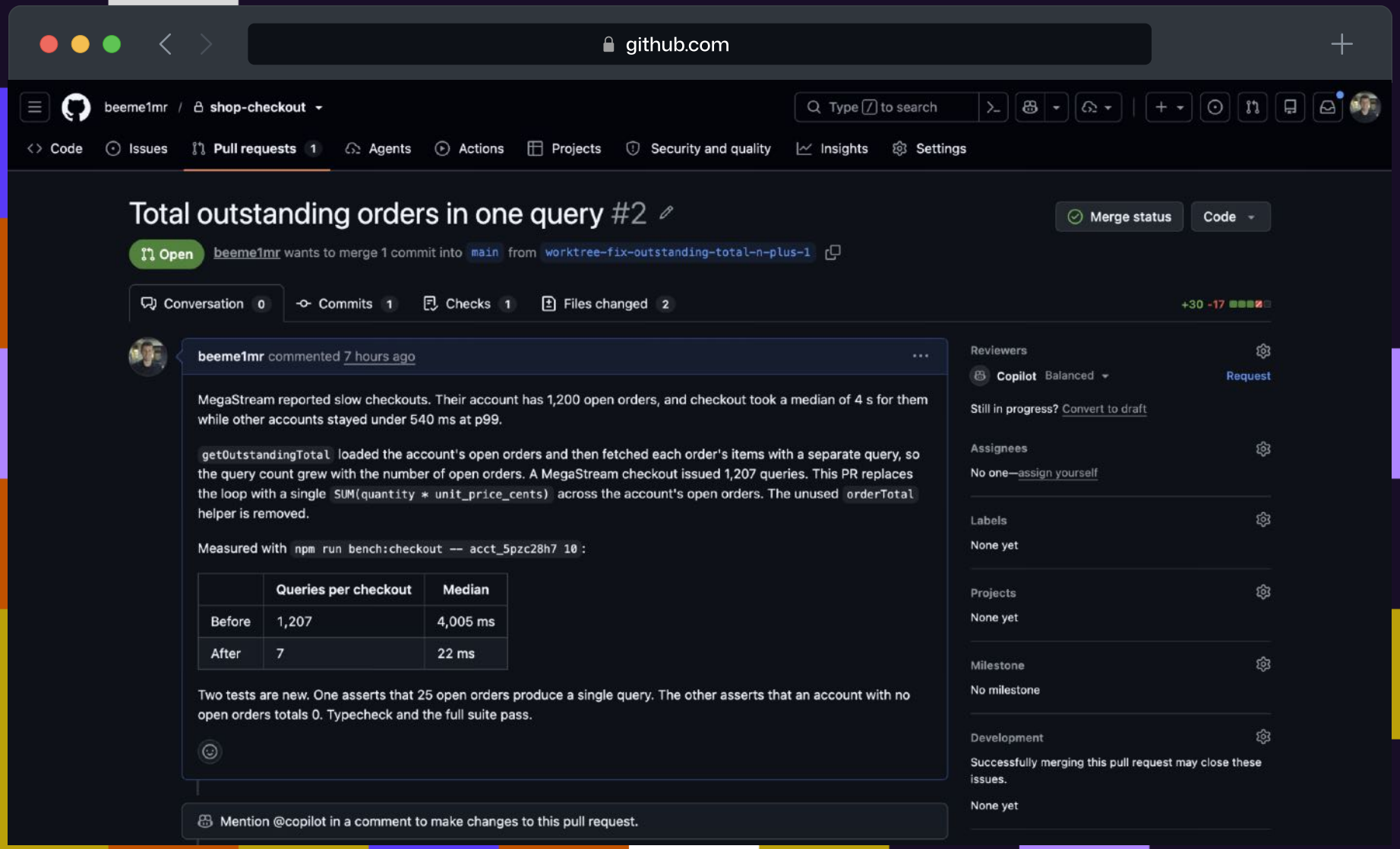
- Every checkout calls `getOutstandingTotal`, which runs `Order.findAll` for the account's open orders.
- It then runs `order.getItems()` in a loop, one query per order.
- That is 1 + N queries per checkout. MegaStream has 1,200 open orders, so about 1,201 queries, while other enterprise accounts have 90–120.
- Latency grows with the number of open orders.

Bluebox discrepancy: Bluebox found the same account as the outlier. It reported no DB child spans for those requests, which I haven't reconciled with the N+1 reading. I'd check trace `c31e3b51c394d50890e5e3208cb463bf` before quoting that to MegaStream.

Suggested fix:

- Replace the loop with a single query that eager-loads `items` (`include: ['items']`).
- Or compute the total with one aggregate `SUM(quantity * unit_price_cents)` joined across the account's open orders.
- The aggregate is better, since it doesn't load rows just to add them up.

And opens a PR



You still own what you ship

That fix was reactive.

A customer found it first.

Prevent it. Detect it. Undo it fast.

Prevent: decide who checks what

Auto-verify

Telemetry checks every deploy against your SLOs.

Human review

Data, auth, schema, or a wide blast radius.

Escalate

SLO burn, or a diagnosis with weak evidence.

Test at scale

Try real data shapes, like 1,200 open orders.

Detect: know before your customers do

SLOS ON EVERY DEPLOY

Compare error rate and p99 before and after each merge.

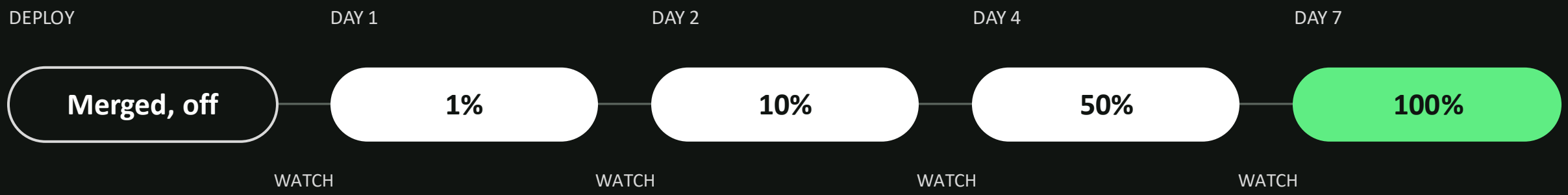
MEASURE WHAT'S IMPORTANT

p99 by account.id, so one customer can't hide in the average.

LET AN AGENT WATCH

A daily Bluebox routine spots outliers like MegaStream and starts the fix before a customer feels it.

Remediate: ship behind a flag



Telemetry goes red at any step? Flip the flag off. Everyone is back on the old code path in seconds, with no redeploy.

Your coding agent knows the code.
Your observability agent knows production.
Pair them.

KEEP GOING

Let’s keep talking

Try Bluebox

bluebox.ai

Visit the Dynatrace / Bluebox booth

Booth [#]

Find me after the session

I’ll be just outside the room.

[QR CODE]
Links to Bluebox



Thank you

[QR CODE]
Session survey QR from the Universe team

How was this session?

We do actually read these. It's how Universe gets better every year.
Scan to review.

